Task 4 – TenderAPP

5. laboratorijska vježba – TDD razvoj novih funkcionalnosti

Osnove verifikacije i validacije softvera

Grupa 1 - Tim 2

Predmet: VVS – TenderAPP projekat

Zadatak: Specificirati i implementirati 2 nove funkcionalnosti primjenom TDD pristupa

Opis zadatka

U okviru 5. laboratorijske vježbe (Task 4) potrebno je proširiti aplikaciju **TenderAPP** sa dvjema novim funkcionalnostima i razviti ih primjenom **Test-Driven Development (TDD)** pristupa.

Zadano:

- Specificirati **dvije nove funkcionalnosti** za aplikaciju.
- Za svaku funkcionalnost prvo napisati testove, a zatim implementirati kod tako da testovi prođu.
- Organizovati tim tako da **svi članovi učestvuju** i u pisanju testova i u implementaciji.
- U izvještaju opisati funkcionalnosti i način njihovog razvijanja u odnosu na TDD ciklus (Red–Green–Refactor).

Funkcionalnost 1 – Sortiranje ponuda po ukupnom skoru

Specifikacija funkcionalnosti

Cilj prve funkcionalnosti je da se za zadani tender omogući **sortiranje svih ponuda** prema polju `UkupanSkor` u opadajućem poretku, tako da se na vrhu liste nalaze ponude sa najvećim bodovima.

Poslovna pravila

- Metoda treba da primi instancu `Tender` i sortira njegovu listu `Ponude`.
- Prije sortiranja za svaku ponudu se poziva `IzracunajSkor(KriterijiTendera)`.
- Ako je tender `null`, baca se `ArgumentNullException`.
- Ako tender nema nijednu ponudu, baca se `TenderException` sa odgovarajućom porukom.

TDD razvoj funkcionalnosti

Korak 1 – Red (neuspjeli test)

Napisali smo unit test **`Test_SortiranjePonuda`** u klasi `TenderServiceTests`. Test:

- kreira pondere (`PonderiKriterija`) i kriterije (`KriterijiTendera`),
- kreira jedan tender i tri ponude sa različitim ulaznim vrijednostima,
- validira ponude i računa im skor,
- poziva `SortirajPonude(tender)`,
- provjerava da je poredak u listi opadajući po `UkupanSkor`.

Pošto metoda u servisu nije postojala, test je inicijalno **padao**.

Korak 2 – Green (minimalna implementacija)

U klasu `TenderService` je dodana sljedeća metoda:

```
public void SortirajPonude(Tender tender)
{
  if (tender == null)
    throw new ArgumentNullException("tender");

  if (tender.Ponude == null || tender.Ponude.Count == 0)
    throw new TenderException("Tender nema ponuda za sortiranje.");

  // Izračun skora za svaku ponudu
  foreach (var ponuda in tender.Ponude)
  {
    ponuda.IzracunajSkor(tender.Kriteriji);
  }

  tender.Ponude = tender.Ponude
    .OrderByDescending(p => p.UkupanSkor)
    .ToList();
}
```

> Nakon implementacije, test **Test_SortiranjePonuda** prolazi uspješno.

Korak 3 – Refactor

Nakon što su testovi postali zeleni, urađeno je minimalno čišćenje:

- provjere `null` vrijednosti i izuzetaka su centralizovane u servisu,
- logika izračunavanja skora ostavljena je u metodi `Ponuda.IzracunajSkor`, čime je očuvana čitljivost i jedinstvena odgovornost.

Funkcionalnost 2 – Čuvanje i učitavanje tendera u JSON fajl

Specifikacija funkcionalnosti

Druga funkcionalnost omogućava trajno čuvanje stanja sistema. Aplikacija treba da:

- sačuva trenutnu listu tendera u JSON fajl na zadatoj putanji,
- učita tender(e) iz postojećeg JSON fajla i popuni internu kolekciju u servisu.

Poslovna pravila

- Metoda `SacuvajTenderJSON(path)` serijalizuje listu tendera u JSON format i upisuje je u fajl.
- Metoda `UcitajTenderJSON(path)` čita JSON fajl, deserijalizuje ga u listu `Tender` objekata i zamjenjuje internu listu.
- Ako fajl ne postoji, baca se `FileNotFoundException`.
- Struktura podataka nakon učitavanja mora odgovarati originalnim objektima (naziv, opis, kriteriji, ponderi...).

TDD razvoj funkcionalnosti

Korak 1 – Red (neuspjeli test)

Napisali smo test **`JsonUpisICitanje_ShouldWorkCorrectly`** u klasi `TenderServiceTests`. Test:

- kreira `TenderService` i jedan tender uz pripadajuće kriterije i pondere,
- poziva `SacuvajTenderJSON(filePath)` za privremenu putanju,
- kreira novi servis, poziva `UcitajTenderJSON(filePath)`,
- provjerava da postoji tačno jedan tender i da su svi ključni podaci isti kao u originalu.

> Pošto metode u servisu nisu bile implementirane, test je isprva padao – klasična RED faza u TDD ciklusu.

Korak 2 – Green (minimalna implementacija)

U interfejs `ITenderService` dodane su metode:

```
void SacuvajTenderJSON(string path);
void UcitajTenderJSON(string path);
```

A u realizaciji servisa `TenderService` implementacija izgleda ovako:

```
public void SacuvajTenderJSON(string path)
{
  var options = new JsonSerializerOptions { WriteIndented = true };
  string json = JsonSerializer.Serialize(tenderi, options);
  File.WriteAllText(path, json);
}

public void UcitajTenderJSON(string path)
{
  if (!File.Exists(path))
    throw new FileNotFoundException("JSON fajl nije pronađen", path);

  string json = File.ReadAllText(path);
  tenderi = JsonSerializer.Deserialize<List<Tender>>(json) ?? new
List<Tender>();
}
```

> Nakon ove implementacije test za čuvanje i učitavanje JSON-a prolazi, čime je GREEN faza završena.

Korak 3 – Refactor (podrška modela za deserializaciju)

Tokom testiranja pojavile su se problemi sa deserializacijom kompleksnih tipova (`Tender`, `PonderiKriterija`, `Ponuda`). Problem je bio u tome što `System.Text.Json` nije mogao da nađe odgovarajuće konstruktore.

Rješenje je bilo dodavanje **praznih (parameterless) konstruktora** u modele:

- `Tender()`
- `PonderiKriterija()`
- `Ponuda()` – posebno zbog toga što je postojala verzija konstruktora sa parametrima koji se nisu mapirali 1-na-1 na property-je (npr. `tender`, `rok`).

Nakon toga, JSON deserializacija je radila ispravno, a svi testovi su ostali zeleni.

Integracija u korisnički interfejs

Kako bi funkcionalnost bila dostupna korisniku, dodan je novi podmeni u `MenuView` i odgovarajuća logika u `Program.cs`:

- Opcija **1. Sačuvaj tendere u JSON fajl**
- Opcija **2. Učitaj tendere iz JSON fajla**

Ove opcije internim pozivima delegiraju posao na `TenderController`, a zatim na `TenderService`, zadržavajući jasan razdvajanje UI i poslovne logike.

TDD i timski rad

Organizacija tima

Task 4 je izveden tako da svi članovi tima aktivno učestvuju u TDD ciklusu:

- Jedan član je inicijalno pisao testove za nove funkcionalnosti (`TenderServiceTests`).
- Drugi član je radio na implementaciji metoda u servisnom sloju.
- Treći član je radio na integraciji u konzolni UI (`MenuView`, `TenderController`, `Program.cs`).
- Svi članovi su učestvovali u analizi grešaka (posebno problema sa JSON deserializacijom) i refaktoringu modela.

TDD ciklus koji smo pratili:

1. **Red** – pisanje testova koji u startu padaju.
2. **Green** – implementacija minimalnog koda u servisu tako da testovi prođu.
3. **Refactor** – čišćenje koda, uvođenje parameterless konstruktora i sitne dorade bez rušenja testova.

Zaključak

Kroz Task 4 aplikacija **TenderAPP** je proširena sa dvije praktične funkcionalnosti:

- sortiranje ponuda po ukupnom skoru (evaluacija ponuda),
- čuvanje i učitavanje tendera u JSON fajl (persistencija podataka).

> TDD pristup je omogućio da se obje funkcionalnosti razvijaju uz **kontinuirnu provjeru ispravnosti**, a testovi ostaju osnova za buduće izmjene i refaktorisanje sistema.

Univerzitet u Sarajevu
Elektrotehnički fakultet Sarajevo – Odsjek za računarstvo i informatiku

VVS – 5. laboratorijska vježba (Task 4)
© 2025